

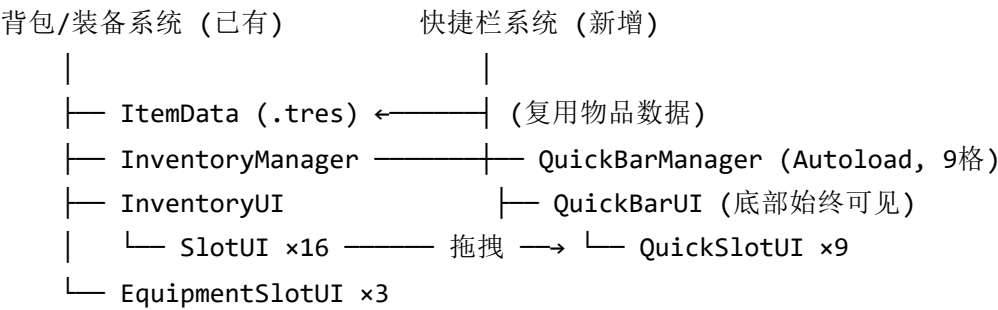
教程：快捷物品栏系统

架构概览

本教程分两大部分：

- **第一部分：思路引导** — 引导你自己推导出实现方案，只对全新概念（拖拽放置）给出代码
- **第二部分：完整操作参考** — 和上一份教程风格一致的详细步骤

系统关系图



文件清单

文件	作用	新建/修改
scripts/quickbar_manager.gd	快捷栏管理单例	新建
scenes/ui/quick_slot_ui.tscn	快捷栏格子组件	新建
scenes/ui/quick_slot_ui.gd	格子控制脚本	新建
scenes/ui/quickbar_ui.tscn	快捷栏主面板	新建
scenes/ui/quickbar_ui.gd	面板控制脚本	新建
scenes/ui/slot_ui.gd	背包格子（新增拖拽）	修改
scenes/main.tscn	主场景（挂载面板）	修改
project.godot	项目设置	修改

第一部分：思路引导

先想一想

你已经完成了背包/装备系统，这个新系统的本质是什么？

快捷栏 = 9个快速入口，指向背包里的物品

它不是第二背包，更**不是**另一套数据仓库

你要做的，是建立一套 9 格的"快捷引用"系统——每格指向背包里的一个物品，按数字键触发对应的装备或使用动作。

在往下看之前，试着回答这几个问题：

引导1：架构层

上一份教程的架构是：**ItemData** → **InventoryManager** → **UI (SlotUI + EquipmentSlotUI)**

快捷栏的架构能不能复用这个模式？去写下三层的角色分别是什么。

答案只有一个全新概念：背包格子如何"变成"可以被拖拽的源

引导2：数据层

看 **InventoryManager** 的结构——它用 `backpack: Array` 存储 16 格背包，用 `equipment: Dictionary` 存储 3 个装备槽。

你的 **QuickBarManager** 需要什么数据结构？

- Array 还是 Dictionary？
- 长度多少？
- 需要什么信号？

试着在脑子里画出来。

引导3：UI组件

SlotUI 是携带了 `set_item(item)` + 唯一名称占位 + 双击信号的独立组件。

QuickSlotUI 有什么不同？

对比项	SlotUI (背包)	QuickSlotUI (快捷栏)
点击事件	双击装备	需要什么？
拖拽	无	需要什么来接受物品？
尺寸	48×48	你觉得多大合适？

QuickBarUI 和 InventoryUI 又有什么不同？

- InventoryUI 按 B 展现/隐藏
- QuickBarUI 应该是怎样的？一直显示还是有触发条件？

引导4：交互输入

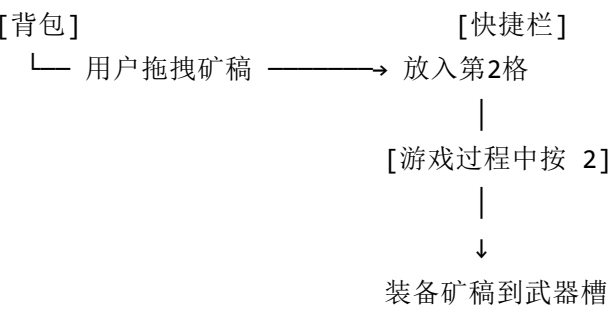
InventoryUI 用 `_input` 监听 `toggle_inventory` (B 键)。

QuickBarUI 同样用 `_input` 监听数字键 1-9。

这里的输入映射你打算怎么注册？去项目设置看上一份教程怎么注册 `toggle_inventory` 的。

引导5：交互流程

这是整个系统最有意思的部分。思考这个流程：



在你设计数据的时候想想：

1. 快捷栏里存的是什么？是**物品本体**还是**引用**？
2. 如果在背包里已经把矿稿装备上了，快捷栏第2格会怎么样？
3. QuickBarManager 怎么"使用"一个快捷栏物品？
 - 如果是武器/护甲/电池 → 需要什么操作？
 - 如果是消耗品 → 会有什么不同？

新概念：拖拽放置（附代码）

这部分是你没接触过的，所以直接给出代码和解释。

Godot 的拖拽系统由三个虚拟方法构成：

```

# 第一步：拖出方调用 _get_drag_data
# 作用：启动拖拽，返回被拖拽的数据
# 调用时机：鼠标按下并开始拖动时
func _get_drag_data(_at_position: Vector2) -> Variant:
    # 如果当前格子没有物品，不能拖拽
    if current_item == null:
        return null

    # 创建一个预览图，跟随鼠标
    var preview := TextureRect.new()
    preview.texture = current_item.icon
    preview.custom_minimum_size = Vector2(48, 48)
    preview.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    preview.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    set_drag_preview(preview)

    # 返回拖拽数据
    return {"item": current_item, "source": "backpack"}

# 第二步：接受方调用 _can_drop_data
# 作用：判断是否能接受这个拖拽数据
# 调用时机：拖拽经过该控件时
func _can_drop_data(_at_position: Vector2, data: Variant) -> bool:
    # 如果数据是一个带"item"键的字典，就可以接受
    return data is Dictionary and data.has("item")

# 第三步：接受方调用 _drop_data
# 作用：处理放置事件，将物品设置到本格子
# 调用时机：用户松手时
func _drop_data(_at_position: Vector2, data: Variant) -> void:
    var item := data["item"] as ItemData
    if item != null:
        QuickBarManager.set_slot(slot_index, item)

```

关键理解：

- 三个方法都是 override，Godot 自动调用
- 你无需手动连接任何信号
- set_drag_preview() 创建拖拽时跟随鼠标的预览图
- 返回的数据是什么都行（字典、对象、引用），只要收发双方约定一致

提示清单

在开始实现之前，再看一遍这些提示，看能不能补全你自己的实现方案：

1. **QuickBarManager** 是否需要 `class_name`？会不会和 `autoload` 冲突？（回想一下 `InventoryManager` 的教训）
2. **QuickSlotUI** 是否需要 `class_name`？
3. `QuickBarUI` 的 `_input` 监听数字键时，用 `event.is_action_pressed("quickbar_1")` 还是 `event.keycode == KEY_1`？哪个更推荐？(Hint: input mapping)
4. 数字键 1-9 对应的输入动作叫什么？需要注册几个输入映射？
5. `QuickBarUI` 在场景树中应该放在哪里？（看看 `InventoryUI` 在哪）
6. **SlotUI 需要一个 `current_item` 成员变量**来存当前格子的物品引用——因为 `_get_drag_data` 需要知道拖拽什么东西。但注意：`set_item()` 只在背包刷新时调用，所以 `current_item` 可以用来追踪当前物品。
7. 快捷栏用完后，物品是从背包移除还是保留？(思考：`quick slot` 指向 `backpack`，使用后装备到对应装备栏，背包中的该物品照常被 `InventoryManager` 管理)
8. 如果拖拽设置快捷栏时，该槽位已有物品，是覆盖还是忽略？

第二部分：完整操作参考

第一步：创建 `QuickBarManager` 自动加载单例

1.1 新建脚本

1. 右键 `scripts` 文件夹 → **新建** → **脚本...**
2. 配置：
 - **继承**： `Node`
 - **路径**： `res://scripts/quickbar_manager.gd`
3. 点击 **创建**

1.2 编写代码

```
extends Node
```

```
# 信号：快捷栏发生变化时通知 UI 刷新
```

```
signal quickbar_changed
```

```
const QUICKBAR_SIZE := 9
```

```
# 快捷栏：存的是 ItemData，空位为 null
```

```
var quickbar: Array = []
```

```
func _ready() -> void:
```

```
    quickbar.resize(QUICKBAR_SIZE)
```

```
func set_slot(index: int, item: ItemData) -> void:
```

```
    if index >= 0 and index < QUICKBAR_SIZE:
```

```
        quickbar[index] = item
```

```
        quickbar_changed.emit()
```

```
func clear_slot(index: int) -> void:
```

```
    set_slot(index, null)
```

```
func use_slot(index: int) -> void:
```

```
    var item := quickbar[index] as ItemData
```

```
    if item == null:
```

```
        return
```

```
    match item.item_type:
```

```
        ItemData.ItemType.WEAPON, ItemData.ItemType.ARMOR, ItemData.ItemType.BATTERY:
```

```
            # 在背包中找到该物品并装备
```

```
            for i in range(InventoryManager.BACKPACK_SIZE):
```

```
                if InventoryManager.backpack[i] == item:
```

```
                    InventoryManager.equip(i)
```

```
                    return
```

```
        ItemData.ItemType.CONSUMABLE:
```

```
            # 消耗品使用逻辑（预留）
```

```
            pass
```

1.3 注册为自动加载

1. 菜单 **项目** → **项目设置...** → **自动加载** 标签页
2. **路径** → 选 `res://scripts/quickbar_manager.gd`
3. **节点名称** → `QuickBarManager`
4. 点击 **添加**

注意：不要加 `class_name`，`autoload` 已经是全局可访问的

第二步：创建快捷栏格子组件 QuickSlotUI

2.1 创建场景

1. 右键 `scenes/ui` → **新建** → **场景**
2. 根节点选 `PanelContainer`，改名为 `QuickSlotUI`
3. 保存为 `res://scenes/ui/quick_slot_ui.tscn`

2.2 搭建节点

```
QuickSlotUI (PanelContainer)
├─ VBoxContainer
│   ├── TextureRect    ← 物品图标
│   └─ Label           ← 快捷键编号
```

操作：

1. 主题样式跟 `SlotUI` 类似：
 - **Bg Color**： `#333333`
 - **Border Width**： `2px`，颜色 `#555555`
 - **Corner Radius**： `4`
2. `TextureRect`：
 - 命名为 `IconPlaceholder`
 - **Expand Mode**： `Ignore Size`
 - **Stretch Mode**： `Keep Aspect Centered`
 - **Min Size**： `36×36`（比背包格子小一些）
 - 设为唯一名称（%）
3. `Label`：
 - 命名为 `SlotNumber`
 - **Horizontal Alignment**： `Center`
 - 显示快捷键数字（1-9）

2.3 创建脚本 res://scenes/ui/quick_slot_ui.gd

```
class_name QuickSlotUI
extends PanelContainer

signal slot_used(slot_index: int)

var slot_index: int = -1
var current_item: ItemData = null

func set_item(item: ItemData, number_text: String) -> void:
    current_item = item
    if item == null:
        %IconPlaceholder.texture = null
        %SlotNumber.text = number_text
    else:
        %IconPlaceholder.texture = item.icon
        %SlotNumber.text = item.item_name

# 拖拽放置：判断是否可以接受
func _can_drop_data(_at_position: Vector2, data: Variant) -> bool:
    return data is Dictionary and data.has("item")

# 拖拽放置：处理放置
func _drop_data(_at_position: Vector2, data: Variant) -> void:
    var item := data["item"] as ItemData
    if item != null:
        QuickBarManager.set_slot(slot_index, item)
```

第三步：修改背包格子 SlotUI 支持拖拽

打开 scenes/ui/slot_ui.gd，新增 current_item 成员和 _get_drag_data 方法：

```

class_name SlotUI
extends PanelContainer

signal slot_double_clicked(slot_index: int)

var slot_index: int = -1
var current_item: ItemData = null      # ← 新增：保存当前物品引用

func set_item(item: ItemData) -> void:
    current_item = item                # ← 新增：保存引用
    if item == null:
        %IconPlaceholder.texture = null
        %ItemName.text = ""
    else:
        %IconPlaceholder.texture = item.icon
        %ItemName.text = item.item_name

func _gui_input(event: InputEvent) -> void:
    if event is InputEventMouseButton and event.double_click:
        slot_double_clicked.emit(slot_index)

# 新增：拖拽开始
func _get_drag_data(_at_position: Vector2) -> Variant:
    if current_item == null:
        return null

    var preview := TextureRect.new()
    preview.texture = current_item.icon
    preview.custom_minimum_size = Vector2(48, 48)
    preview.expand_mode = TextureRect.EXPAND_IGNORE_SIZE
    preview.stretch_mode = TextureRect.STRETCH_KEEP_ASPECT_CENTERED
    set_drag_preview(preview)

    return {"item": current_item, "source": "backpack"}

```

第四步：创建快捷栏主面板 QuickBarUI

4.1 创建场景

1. 右键 `scenes/ui` → **新建** → **场景**
2. 根节点选 `PanelContainer`，改名为 `QuickBarUI`
3. 保存为 `res://scenes/ui/quickbar_ui.tscn`

4.2 搭建节点

```
QuickBarUI (PanelContainer)
├─ MarginContainer (留边距)
│   └─ HBoxContainer "QuickBarRow"
│       ├── QuickSlotUI (索引0, 数字1)
│       ├── QuickSlotUI (索引1, 数字2)
│       ├── ...
│       └─ QuickSlotUI (索引8, 数字9)
```

操作：

1. **根节点样式**：简单半透明深色背景
 - **Bg Color**： `#1a1a1a88`
 - **Border Width**： 1，颜色 `#444444`
 - **Corner Radius**： 6
 - **Content Margin**： 四边各 8
2. 根节点 → 添加子节点 `MarginContainer`
3. `MarginContainer` → 添加子节点 `HBoxContainer`，命名为 `QuickBarRow`
4. **Theme Overrides** → **Constants**： `H Separation=4`
5. 把 `quick_slot_ui.tscn` 从 `FileSystem` 拖到 `QuickBarRow` 下，拖 **9 次**
6. 逐一选中每个实例，检查器设 `slot_index`：
 - `Slot Key = 0, 1, 2, ..., 8`

4.3 创建脚本 `res://scenes/ui/quickbar_ui.gd`

```
extends PanelContainer
```

```
func _ready() -> void:  
    _refresh_all()  
    QuickBarManager.quickbar_changed.connect(_refresh_all)
```

```
func _input(event: InputEvent) -> void:  
    for i in 9:  
        if event.is_action_pressed("quickbar_" + str(i + 1)):  
            QuickBarManager.use_slot(i)  
            get_viewport().set_input_as_handled()  
            return
```

```
func _refresh_all() -> void:  
    var row := $MarginContainer/QuickBarRow  
    for i in row.get_child_count():  
        var slot := row.get_child(i) as QuickSlotUI  
        if i < QuickBarManager.quickbar.size():  
            slot.slot_index = i  
            slot.set_item(QuickBarManager.quickbar[i], str(i + 1))
```

第五步：注册输入动作 + 集成到主场景

5.1 注册 9 个数字键输入映射

1. 项目 → 项目设置 → 输入映射 标签页
2. 分别添加 9 个动作：quickbar_1、quickbar_2、... quickbar_9
3. 每个动作绑定对应的数字键（1-9）

5.2 将面板挂载到主场景

打开 scenes/main.tscn：

1. 在已有 CanvasLayer 下，把 quickbar_ui.tscn 从 FileSystem 拖进去
2. 调整位置：
 - 选中 QuickBarUI 根节点
 - 在检查器 → Layout → **Full Rect**（或手动设 Anchor Preset：Bottom Wide）

- 让它固定在屏幕底部居中

提示：用 Layout 锚点面板的 **Bottom Wide** 预设一键搞定底部居中

5.3 设置 QuickBarUI 的布局锚点

由于 QuickBarUI 在 CanvasLayer 下，需要确保它始终显示在屏幕底部：

- **Layout** → **Anchors**：Left=0, Right=1, Top=1, Bottom=1
- 或者直接用 Layout 菜单 → **Bottom Wide**
- 微调：设 `margin_top = -100`（从底部向上 100px 开始）

第六步：运行测试

操作	预期效果
启动游戏	快捷栏显示在屏幕底部，9个空格标有1-9
按 B 打开背包	看见矿稿在第一格
从背包拖拽矿稿到快捷栏第2格	快捷栏第2格显示矿稿
按数字键 2	矿稿被装备到武器槽，快捷栏第2格清空
再次拖拽矿稿到快捷栏第2格	再次设置成功
按数字键 1（空）	无反应（不会报错）
拖拽空背包格子	无法拖拽

常见问题排查

现象	可能原因	解决
拖拽时没有预览图	<code>_get_drag_data</code> 返回 <code>null</code>	检查 <code>current_item</code> 是否被正确赋值为当前物品
拖拽无法放到快捷栏	<code>mouse_filter</code> 拦截了拖拽	确保 QuickSlotUI 的 Mouse → Filter 不是 Ignore

现象	可能原因	解决
按数字键没反应	输入动作未注册	确认 quickbar_1 ~ quickbar_9 已在输入映射注册
数字键被其他功能拦截	场景中其他节点的 _input 优先处理了	get_viewport().set_input_as_handled() 确保只处理一次
快捷栏不在底部	锚点没设置	用 Layout → Bottom Wide 预设
快捷栏在 InventoryUI 前面/后面	场景树顺序问题	调整 QuickBarUI 在 CanvasLayer 中的顺序 (拖拽节点顺序)
拖拽后物品显示没更新	信号未连接或没触发	检查 QuickBarManager.quickbar_changed 连接
_get_drag_data 报错	SlotUI 中缺少 current_item 变量	确认在 set_item() 中赋值 current_item = item